

LAPORAN PRAKTIKUM DAN TUGAS

STRUKTUR DATA

“Insertion Sort,Bubble Sort, dan Selection Sort”

DISUSUN OLEH:

Aliyatar Rafi Ahmad

2511533031

DOSEN PENGAMPU:

Dr. Wahyudi, S.T, M.T

ASISTEN PRAKTIKUM:

Sherly Sukmadira



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum Struktur Data pekan ke-7 ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai bentuk dokumentasi sekaligus pemahaman terhadap materi praktikum yang telah dilakukan mengenai algoritma pengurutan data (*sorting*), yaitu *Insertion Sort*, *Selection Sort*, dan *Bubble Sort* menggunakan bahasa pemrograman Java.

Dalam praktikum ini, penulis mempelajari konsep dasar pengurutan data beserta cara kerja masing-masing algoritma *sorting*. Selain memahami teori, penulis juga mengimplementasikan algoritma tersebut ke dalam program sehingga dapat mengetahui proses pengurutan data secara lebih jelas dan terstruktur. Melalui praktikum ini diharapkan kemampuan dalam memahami logika algoritma dan pemrograman dapat semakin meningkat.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan baik dari segi penulisan maupun isi materi. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun agar laporan ini dapat menjadi lebih baik di masa mendatang. Semoga laporan praktikum ini dapat memberikan manfaat bagi penulis maupun pembaca.

Padang, 20 Mei 2026

Aliyatar Rafi Ahmad

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan.....	1
1.3 Manfaat.....	2
BAB II PEMBAHASAN	3
2.1 Insertion Sort.....	3
2.2 Selection Sort.....	3
2.3 Bubble Sort	3
2.4 Implementasi dalam Java.....	4
2.4.1 InsertionSort_2511533031	4
2.4.2 SelectionSort_2511533031	5
2.4.3 BubbleSort_2511533031	6
2.4.4 InsertionGUI_2511533031	7
BAB III PENUTUP	12
3.1 Kesimpulan	12
DAFTAR PUSTAKA.....	13
LAPORAN TUGAS	14

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pengurutan data (*sorting*) merupakan salah satu konsep penting dalam struktur data dan algoritma. Proses pengurutan digunakan untuk menyusun data ke dalam urutan tertentu, seperti dari kecil ke besar maupun sebaliknya, sehingga data menjadi lebih mudah dibaca, dicari, dan diolah. Dalam dunia pemrograman, algoritma *sorting* sering digunakan untuk meningkatkan efisiensi pengolahan data pada berbagai aplikasi.

Pada praktikum pekan ke-7 ini dipelajari tiga metode pengurutan sederhana, yaitu *Insertion Sort*, *Selection Sort*, dan *Bubble Sort*. Ketiga algoritma tersebut memiliki cara kerja yang berbeda dalam melakukan proses pengurutan data. Dengan mempelajari algoritma *sorting*, mahasiswa dapat memahami logika dasar pengurutan data serta mengetahui kelebihan dan kekurangan dari masing-masing metode.

Selain memahami teori, praktikum ini juga bertujuan untuk mengimplementasikan algoritma *sorting* menggunakan bahasa pemrograman Java. Melalui implementasi program, proses pengurutan data dapat diamati secara langsung sehingga membantu meningkatkan kemampuan analisis logika dan pemrograman.

1.2 Tujuan

1. Memahami konsep dasar algoritma pengurutan data (*sorting*).
2. Memahami cara kerja algoritma *Insertion Sort*, *Selection Sort*, dan *Bubble Sort*.
3. Mengetahui proses pengurutan data dari awal hingga data terurut.
4. Memahami penerapan *Insertion Sort* menggunakan Graphical User Interface (GUI).

1.3 Manfaat

1. Menambah pemahaman mengenai struktur data dan algoritma sorting.
2. Melatih kemampuan berpikir logis dalam menyelesaikan masalah pemrograman.
3. Membantu memahami perbedaan cara kerja beberapa metode pengurutan data.

BAB II

PEMBAHASAN

2.1 Insertion Sort

Insertion Sort adalah algoritma pengurutan sederhana yang bekerja dengan cara menyisipkan setiap elemen dari bagian data yang belum terurut ke posisi yang benar pada bagian data yang sudah terurut. Cara kerjanya mirip seperti saat menyusun kartu remi di tangan, yaitu dengan memisahkan kartu yang sudah tersusun dan yang belum tersusun, kemudian mengambil satu kartu dan menempatkannya pada posisi yang sesuai.

Proses *Insertion Sort* dimulai dengan menganggap elemen pertama sudah terurut. Selanjutnya, elemen kedua dibandingkan dengan elemen pertama, lalu ditukar jika nilainya lebih kecil. Proses kemudian dilanjutkan ke elemen berikutnya dengan membandingkan dan menempatkannya pada posisi yang tepat hingga seluruh data berhasil diurutkan.

2.2 Selection Sort

Selection Sort adalah algoritma pengurutan berbasis perbandingan yang bekerja dengan cara memilih elemen terkecil atau terbesar dari bagian data yang belum terurut, kemudian menukarnya dengan elemen pada posisi awal bagian tersebut.

Proses dimulai dengan mencari nilai terkecil pada seluruh data lalu menukarnya dengan elemen pertama sehingga posisi pertama menjadi terurut. Setelah itu, algoritma mencari nilai terkecil berikutnya dari sisa data yang belum terurut dan menukarnya dengan elemen kedua. Langkah ini dilakukan secara berulang sampai seluruh elemen berada pada posisi yang benar.

2.3 Bubble Sort

Bubble Sort adalah algoritma pengurutan yang bekerja dengan cara membandingkan dua elemen yang berdekatan, kemudian menukarnya apabila

urutannya tidak sesuai. Proses ini dilakukan secara berulang hingga seluruh data berada dalam urutan yang benar.

Algoritma ini dinamakan *Bubble Sort* karena pada setiap proses pengulangan, elemen dengan nilai terbesar akan bergerak ke bagian akhir array seperti gelembung udara yang naik ke permukaan air.

2.4 Implementasi dalam Java

Pada praktikum pekan ke-5 telah dibuat beberapa kode program yang berkaitan dengan implementasi struktur data **Sorting** dalam bahasa pemrograman Java.

2.4.1 InsertionSort_2511533031

```

1. package pekan7_2511533031;
2.
3. public class InsertionSort_2511533031 {
4.     public static void insertionSort(int[] arr_3031) {
5.         int n_3031 = arr_3031.length;
6.         for (int i_3031 = 1; i_3031 < n_3031; i_3031++) {
7.             int key_3031 = arr_3031[i_3031];
8.             int j_3031 = i_3031 - 1;
9.             while (j_3031 >= 0 && arr_3031[j_3031] >
key_3031) {
10.                 arr_3031[j_3031 + 1] = arr_3031[j_3031];
11.                 j_3031--;
12.             }
13.             arr_3031[j_3031 + 1] = key_3031;
14.         }
15.     }
16.     public static void main(String[] args) {
17.         int arr_3031[] = { 23, 78, 45, 8, 32, 56, 1 };
18.         int n_3031 = arr_3031.length;
19.         System.out.printf("array yang belum terurut:\n");
20.         for (int i_3031 = 0; i_3031 < n_3031; i_3031++)
21.             System.out.print(arr_3031[i_3031] + " ");
22.         System.out.println("");
23.         insertionSort(arr_3031);
24.         System.out.printf("array yang terurut:\n");
25.         for (int i_3031 = 0; i_3031 < n_3031; i_3031++)
26.             System.out.print(arr_3031[i_3031] + " ");
27.         System.out.println("");
28.     }
29. }
30. }

```


Program ini digunakan untuk mengimplementasikan algoritma *Insertion Sort* pada array integer. Method `insertionSort()` bekerja dengan cara mengambil satu elemen sebagai key, lalu membandingkannya dengan elemen sebelumnya hingga ditemukan posisi yang sesuai. Jika ada elemen yang lebih besar, maka elemen tersebut akan digeser ke kanan.

Pada method `main()`, program menampilkan array sebelum dan sesudah proses pengurutan dilakukan. Setelah method `insertionSort()` dipanggil, data yang awalnya belum terurut akan tersusun secara ascending menggunakan algoritma *Insertion Sort*.

2.4.2 SelectionSort_2511533031

```

1. package pekan7_2511533031;
2.
3. public class selectionSort_2511533031 {
4.     public static void selectionSort_3031(int[] arr_3031) {
5.         int n_3031 = arr_3031.length;
6.         for (int i_3031 = 0; i_3031 < n_3031; i_3031++) {
7.             int minIndex_3031 = i_3031;
8.             for (int j_3031 = i_3031 + 1; j_3031 < n_3031;
9.                 j_3031++) {
10.                 if (arr_3031[j_3031] <
11.                     arr_3031[minIndex_3031]) {
12.                     minIndex_3031 = j_3031;
13.                 }
14.             }
15.             int temp_3031 = arr_3031[i_3031];
16.             arr_3031[i_3031] = arr_3031[minIndex_3031];
17.             arr_3031[minIndex_3031] = temp_3031;
18.         }
19.     }
20.     public static void main(String[] args) {
21.         int arr_3031[] = {23, 78, 45, 8, 32, 56, 1};
22.         int n_3031 = arr_3031.length;
23.         System.out.printf("array yang belum terurut:\n");
24.         for (int i_3031 = 0; i_3031 < n_3031; i_3031++)
25.             System.out.print(arr_3031[i_3031] + " ");
26.         System.out.println("");
27.         selectionSort_3031(arr_3031);
28.         System.out.printf("array yang terurut:\n");
29.         for (int i_3031 = 0; i_3031 < n_3031; i_3031++)
30.             System.out.print(arr_3031[i_3031] + " ");
31.         System.out.println("");
32.     }
33. }

```

Program ini digunakan untuk mengimplementasikan algoritma *Selection Sort* pada array integer. Method `selectionSort_3031()` bekerja dengan cara mencari nilai terkecil pada bagian array yang belum terurut, kemudian menukarnya dengan elemen pada posisi awal. Proses ini dilakukan berulang hingga seluruh data tersusun secara ascending.

Pada method `main()`, program menampilkan isi array sebelum proses sorting dilakukan, kemudian memanggil method `selectionSort_3031()` untuk mengurutkan data. Setelah proses selesai, array yang sudah terurut ditampilkan kembali sebagai hasil dari algoritma *Selection Sort*.

2.4.3 BubbleSort_2511533031

```

1. package pekan7_2511533031;
2.
3. public class BubleSort_2511533031 {
4.     public static void bubbleSort_3031(int[] arr_3031) {
5.         int n_3031 = arr_3031.length;
6.         for (int i_3031 = 0; i_3031 < n_3031; i_3031++) {
7.             for (int j_3031 = 0; j_3031 < n_3031 - i_3031 -
8. 1; j_3031++) {
9.                 if (arr_3031[j_3031] > arr_3031[j_3031 + 1])
10. {
11.                     int temp_3031 = arr_3031[j_3031];
12.                     arr_3031[j_3031] = arr_3031[j_3031 + 1];
13.                     arr_3031[j_3031 + 1] = temp_3031;
14.                 }
15.             }
16.         }
17.     }
18.     public static void main(String[] args) {
19.         int arr_3031[] = {23, 78, 45, 8, 32, 56, 1};
20.         int n_3031 = arr_3031.length;
21.         System.out.print("array yang belum terurut:\n");
22.         for (int i_3031 = 0; i_3031 < n_3031; i_3031++)
23.             System.out.print(arr_3031[i_3031] + " ");
24.         System.out.println("");
25.         bubbleSort_3031(arr_3031);
26.         System.out.print("array yang terurut menggunakan
27. BubbleSort:\n");
28.         for (int i_3031 = 0; i_3031 < n_3031; i_3031++)
29.             System.out.print(arr_3031[i_3031] + " ");
30.         System.out.println("");
31.     }
32. }

```

Program ini digunakan untuk mengimplementasikan algoritma *Bubble Sort* pada array integer. Method `bubbleSort_3031()` bekerja dengan cara membandingkan dua elemen yang berdekatan, lalu menukarnya apabila urutannya salah. Proses pertukaran dilakukan secara berulang hingga seluruh data tersusun secara ascending.

Pada method `main()`, program menampilkan array sebelum proses pengurutan dilakukan, kemudian memanggil method `bubbleSort_3031()` untuk mengurutkan data. Setelah proses selesai, array yang sudah terurut ditampilkan kembali sebagai hasil dari algoritma *Bubble Sort*.

2.4.4 InsertionGUI_2511533031

```

1. package pekan7_2511533031;
2.
3. import java.awt.BorderLayout;
4. import java.awt.Color;
5. import java.awt.Dimension;
6. import java.awt.FlowLayout;
7. import java.awt.Font;
8.
9. import javax.swing.*;
10. import javax.swing.border.EmptyBorder;
11.
12. public class InsertionGUI_2511533031 extends JFrame {
13.     private static final long serialVersionUID = 1L;
14.     private int[] array_3031;
15.     private JLabel[] labelArray_3031;
16.     private JButton stepButton_3031, resetButton_3031,
        setButton_3031;
17.     private JTextField inputField_3031;
18.     private JPanel panelArray_3031;
19.     private JTextArea stepArea_3031;
20.     private int i_3031 = 1, j_3031;
21.     private boolean sorting_3031 = false;
22.     private int stepCount_3031 = 1;
23.
24.
25.
26.
27. /**
28.  * Create the frame.
29.  */
30. public InsertionGUI_2511533031() {
31.     setTitle("Insertion Sort Langkah per Langkah");
32.     setSize(750, 400);
33.     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
34.     setLocationRelativeTo(null);
35.     setLayout(new BorderLayout());

```

```

36.
37.     // Panel input
38.     JPanel inputPanel_3031 = new JPanel(new FlowLayout());
39.     inputField_3031 = new JTextField(30);
40.     setButton_3031 = new JButton("Set Array");
41.     inputPanel_3031.add(new JLabel("Masukkan angka
    (pisahkan dengan koma):"));
42.     inputPanel_3031.add(inputField_3031);
43.     inputPanel_3031.add(setButton_3031);
44.
45.     // Panel array visual
46.     panelArray_3031 = new JPanel();
47.     panelArray_3031.setLayout(new FlowLayout());
48.
49.     // Panel kontrol
50.     JPanel controlPanel_3031 = new JPanel();
51.     stepButton_3031 = new JButton("Langkah Selanjutnya");
52.     resetButton_3031 = new JButton("Reset");
53.     stepButton_3031.setEnabled(false);
54.     controlPanel_3031.add(stepButton_3031);
55.     controlPanel_3031.add(resetButton_3031);
56.
57.     // Area teks untuk log langkah-langkah
58.     stepArea_3031 = new JTextArea(8, 60);
59.     stepArea_3031.setEditable(false);
60.     stepArea_3031.setFont(new Font("Monospaced",
    Font.PLAIN, 14));
61.     JScrollPane scrollPane_3031 = new
    JScrollPane(stepArea_3031);
62.
63.     // Tambahkan panel ke frame
64.     add(inputPanel_3031, BorderLayout.NORTH);
65.     add(panelArray_3031, BorderLayout.CENTER);
66.     add(controlPanel_3031, BorderLayout.SOUTH);
67.     add(scrollPane_3031, BorderLayout.EAST);
68.
69.     // Event Set Array
70.     setButton_3031.addActionListener(e ->
    setArrayFromInput_3031());
71.
72.     // Event Langkah Selanjutnya\
73.     stepButton_3031.addActionListener(e ->
    performStep_3031());
74.
75.     // Event reset
76.     resetButton_3031.addActionListener(e -> reset_3031 ());
77.     }
78.
79.     private void setArrayFromInput_3031() {
80.         String text = inputField_3031.getText().trim();
81.         if (text.isEmpty()) return;
82.         String[] parts = text.split(",");
83.         array_3031 = new int[parts.length];
84.         try {

```

```

85.         for (int k = 0; k < parts.length; k++) {
86.             array_3031[k] =
Integer.parseInt(parts[k].trim());
87.         }
88.     } catch (NumberFormatException e) {
89.         JOptionPane.showMessageDialog(this, "Masukkan hanya
angka yang dipisahkan "
90.             + "dengan koma!", "Error",
JOptionPane.ERROR_MESSAGE);
91.         return;
92.     }
93.     i_3031 = 1;
94.     stepCount_3031 = 1;
95.     sorting_3031 = true;
96.     stepButton_3031.setEnabled(true);
97.     stepArea_3031.setText("");
98.     panelArray_3031.removeAll();
99.     labelArray_3031 = new JLabel[array_3031.length];
100.    for (int k = 0; k < array_3031.length; k++) {
101.        labelArray_3031[k] = new
JLabel(String.valueOf(array_3031[k]));
102.        labelArray_3031[k].setFont(new Font("Arial",
Font.BOLD, 24));
103.        labelArray_3031[k].setBorder(BorderFactory.createLineBorder(C
olor.BLACK));
104.        labelArray_3031[k].setPreferredSize(new
Dimension(50, 50));
105.        labelArray_3031[k].setHorizontalAlignment(SwingConstants.CENT
ER);
106.        panelArray_3031.add(labelArray_3031[k]);
107.    }
108.    panelArray_3031.revalidate();
109.    panelArray_3031.repaint();
110.    }
111.    private void performStep_3031() {
112.        if (i_3031 < array_3031.length &&
sorting_3031) {
113.            int key_3031 = array_3031[i_3031];
114.            j_3031 = i_3031 - 1;
115.
116.            StringBuilder stepLog_3031 = new
StringBuilder();
117.            stepLog_3031.append("Langkah
").append(stepCount_3031)
118.                .append(": Masukkan
").append(key_3031).append("\n");
119.
120.            while (j_3031 >= 0 && array_3031[j_3031]
> key_3031) {
121.                array_3031[j_3031 + 1] =
array_3031[j_3031];
122.                j_3031--;

```

```

123.         }
124.
125.         array_3031[j_3031 + 1] = key_3031;
126.
127.         updateLabels_3031();
128.         stepLog_3031.append("Hasil: ");
129.
130.         .append(arrayToString(array_3031))
131.         .append("\n\n");
132.
133.         stepArea_3031.append(stepLog_3031.toString());
134.
135.         i_3031++;
136.         stepCount_3031++;
137.
138.         if (i_3031 == array_3031.length) {
139.             sorting_3031 = false;
140.             stepButton_3031.setEnabled(false);
141.             JOptionPane.showMessageDialog(this,
142. "Sorting selesai!");
143.         }
144.     }
145.     private void updateLabels_3031() {
146.         for (int k_3031 = 0; k_3031 <
147. array_3031.length; k_3031++) {
148.             labelArray_3031[k_3031]
149. .setText(String.valueOf(array_3031[k_3031]));
150.         }
151.     }
152.     private void reset_3031() {
153.         inputField_3031.setText("");
154.         panelArray_3031.removeAll();
155.         panelArray_3031.revalidate();
156.         panelArray_3031.repaint();
157.         stepArea_3031.setText("");
158.         stepButton_3031.setEnabled(false);
159.         sorting_3031 = false;
160.         i_3031 = 1;
161.         stepCount_3031 = 1;
162.     }
163.     private String arrayToString(int[] arr_3031) {
164.         StringBuilder sb_3031 = new StringBuilder();
165.
166.         for (int k_3031 = 0; k_3031 <
167. arr_3031.length; k_3031++) {
168.             sb_3031.append(arr_3031[k_3031]);
169.
170.             if (k_3031 < arr_3031.length - 1) {
171.                 sb_3031.append(", ");

```

```

171.         }
172.     }
173.
174.         return sb_3031.toString();
175.     }
176.     public static void main (String [] args) {
177.         SwingUtilities.invokeLater(() -> {
178.             InsertionGUI_2511533031 gui_3029
= new InsertionGUI_2511533031 () ;
179.             gui_3029.setVisible(true);
180.         });
181.     }
182. }

```

Program ini merupakan implementasi algoritma *Insertion Sort* berbasis GUI menggunakan Java Swing. Program memungkinkan pengguna memasukkan data array melalui input teks, kemudian menampilkan proses pengurutan secara bertahap menggunakan tombol “Langkah Selanjutnya”. Setiap langkah sorting ditampilkan secara visual menggunakan label dan juga dicatat pada area teks sehingga proses perpindahan data lebih mudah dipahami.

Class `InsertionGUI_2511533031` menggunakan beberapa komponen GUI seperti `JFrame`, `JPanel`, `JButton`, `JTextField`, `JLabel`, dan `JTextArea`. Method `setArrayFromInput_3031()` digunakan untuk mengambil input array dari pengguna, sedangkan method `performStep_3031()` menjalankan proses *Insertion Sort* langkah demi langkah. Selain itu, terdapat method `updateLabels_3031()` untuk memperbarui tampilan array pada GUI, `reset_3031()` untuk mengembalikan tampilan ke kondisi awal, dan `arrayToString()` untuk mengubah isi array menjadi bentuk string. Dengan adanya GUI, proses sorting menjadi lebih interaktif dan mudah diamati dibandingkan program berbasis console.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma *Insertion Sort*, *Selection Sort*, dan *Bubble Sort* merupakan metode pengurutan data sederhana yang memiliki cara kerja berbeda dalam menyusun data secara ascending. *Insertion Sort* bekerja dengan menyisipkan data ke posisi yang tepat, *Selection Sort* bekerja dengan memilih nilai terkecil pada setiap iterasi, sedangkan *Bubble Sort* bekerja dengan membandingkan dan menukar elemen yang berdekatan.

Pada praktikum ini juga dilakukan implementasi *Insertion Sort* berbasis GUI menggunakan Java Swing. Penggunaan GUI membantu pengguna melihat proses pengurutan data secara lebih interaktif dan visual sehingga lebih mudah dipahami. Melalui praktikum ini, mahasiswa dapat memahami konsep dasar sorting serta mampu mengimplementasikan algoritma pengurutan data menggunakan bahasa pemrograman Java baik berbasis console maupun GUI.

DAFTAR PUSTAKA

- [1] GeeksforGeeks, "Insertion Sort Algorithm," GeeksforGeeks, 24 February 2026. [Online]. Available: <https://www.geeksforgeeks.org/dsa/insertion-sort-algorithm/>. [Accessed 20 Mei 2026].
- [2] GeeksforGeeks, "Selection Sort," GeeksforGeeks, 8 December 2025. [Online]. Available: <https://www.geeksforgeeks.org/dsa/selection-sort-algorithm-2/>. [Accessed 20 Mei 2026].
- [3] Programiz, "Bubble Sort," Programiz, [Online]. Available: https://www.programiz.com/dsa/bubble-sort?utm_source=chatgpt.com. [Accessed 20 Mei 2026].

LAPORAN TUGAS

A. Kode Program

Berikut kode program yang telah dibuat pada tugas praktikum pekan-7:

1. Mahasiswa_2511533031

```
1. package pekan7_2511533031;
2.
3. public class Mahasiswa_2511533031 {
4.
5.     // ===== ATRIBUT =====
6.     private String nama_3031;
7.     private String nim_3031;
8.     private String prodi_3031;
9.
10.    // ===== CONSTRUCTOR =====
11.    public Mahasiswa_2511533031(String nama_3031, String
nim_3031, String prodi_3031) {
12.        this.nama_3031 = nama_3031;
13.        this.nim_3031 = nim_3031;
14.        this.prodi_3031 = prodi_3031;
15.    }
16.
17.    // ===== GETTER =====
18.    public String getNama_3031() {
19.        return nama_3031;
20.    }
21.
22.    public String getNim_3031() {
23.        return nim_3031;
24.    }
25.
26.    public String getProdi_3031() {
27.        return prodi_3031;
28.    }
29.
30.    // ===== SETTER =====
31.    public void setNama_3031(String nama_3031) {
32.        this.nama_3031 = nama_3031;
33.    }
34.
35.    public void setNim_3031(String nim_3031) {
36.        this.nim_3031 = nim_3031;
37.    }
38.
39.    public void setProdi_3031(String prodi_3031) {
40.        this.prodi_3031 = prodi_3031;
41.    }
42.
43.    // ===== toString =====
44.    @Override
45.    public String toString() {
```

```

46.         return nama_3031 + " | " + nim_3031 + " | " +
           prodi_3031;
47.     }
48. }

```

Program ini digunakan untuk merepresentasikan data mahasiswa dalam bentuk class Mahasiswa_2511533031. Class ini memiliki tiga atribut utama, yaitu nama_3031, nim_3031, dan prodi_3031 yang digunakan untuk menyimpan informasi mahasiswa. Constructor pada class digunakan untuk menginisialisasi nilai atribut saat objek dibuat.

Selain itu, class ini juga memiliki method getter dan setter yang berfungsi untuk mengambil serta mengubah nilai atribut. Method toString() digunakan untuk menampilkan data mahasiswa dalam bentuk string sehingga data dapat ditampilkan dengan lebih rapi dan mudah dibaca.

2. SortingGUI_2511533031

```

1. package pekan7_2511533031;
2.
3. import java.awt.*;
4. import java.util.ArrayList;
5. import javax.swing.*;
6. import javax.swing.table.DefaultTableModel;
7.
8. public class SortingGUI_2511533031 extends JFrame {
9.
10.     private static final long serialVersionUID = 1L;
11.
12.     ArrayList<Mahasiswa_2511533031> listMahasiswa_3031 = new
        ArrayList<>();
13.
14.     JTextField txtNama_3031, txtNim_3031, txtProdi_3031;
15.
16.     JButton btnTambah_3031;
17.     JButton btnSort_3031;
18.     JButton btnHapus_3031;
19.     JButton btnReset_3031;
20.
21.     JComboBox<String> comboSort_3031;
22.
23.     JTable tabel_3031;
24.     DefaultTableModel model_3031;
25.
26.     JTextArea areaLog_3031;
27.
28.     public SortingGUI_2511533031() {

```

```

29.
30.     setTitle("Program Sorting Mahasiswa");
31.     setSize(850, 600);
32.     setLocationRelativeTo(null);
33.     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
34.     setLayout(new BorderLayout());
35.
36.     JPanel panelAtas_3031 = new JPanel();
37.     panelAtas_3031.setLayout(new GridLayout(3, 4, 5, 5));
38.
39.     panelAtas_3031.add(new JLabel("Nama"));
40.     txtNama_3031 = new JTextField();
41.     panelAtas_3031.add(txtNama_3031);
42.
43.     panelAtas_3031.add(new JLabel("NIM"));
44.     txtNim_3031 = new JTextField();
45.     panelAtas_3031.add(txtNim_3031);
46.
47.     panelAtas_3031.add(new JLabel("Prodi"));
48.     txtProdi_3031 = new JTextField();
49.     panelAtas_3031.add(txtProdi_3031);
50.
51.     panelAtas_3031.add(new JLabel("Pilih Sorting"));
52.     String[] pilihan_3031 = {"Insertion Sort", "Selection
Sort", "Bubble Sort"};
53.     comboSort_3031 = new JComboBox<>(pilihan_3031);
54.     panelAtas_3031.add(comboSort_3031);
55.
56.     btnTambah_3031 = new JButton("Tambah");
57.     panelAtas_3031.add(btnTambah_3031);
58.
59.     btnHapus_3031 = new JButton("Hapus");
60.     panelAtas_3031.add(btnHapus_3031);
61.
62.     btnSort_3031 = new JButton("Sorting");
63.     panelAtas_3031.add(btnSort_3031);
64.
65.     btnReset_3031 = new JButton("Reset");
66.     panelAtas_3031.add(btnReset_3031);
67.
68.     add(panelAtas_3031, BorderLayout.NORTH);
69.
70.     String[] kolom_3031 = {"No", "Nama", "NIM", "Prodi"};
71.
72.     model_3031 = new DefaultTableModel(kolom_3031, 0);
73.
74.     tabel_3031 = new JTable(model_3031);
75.
76.     JScrollPane scrollTabel_3031 = new
JScrollPane(tabel_3031);
77.
78.     add(scrollTabel_3031, BorderLayout.CENTER);
79.
80.     areaLog_3031 = new JTextArea();

```

```

81.         areaLog_3031.setEditable(false);
82.
83.         JScrollPane scrollLog_3031 = new
84.             JScrollPane(areaLog_3031);
85.         scrollLog_3031.setPreferredSize(new Dimension(100,
86.             180));
87.
88.         add(scrollLog_3031, BorderLayout.SOUTH);
89.
90.         btnTambah_3031.addActionListener(e ->
91.             tambahData_3031());
92.         btnHapus_3031.addActionListener(e -> hapusData_3031());
93.         btnSort_3031.addActionListener(e ->
94.             mulaiSorting_3031());
95.         btnReset_3031.addActionListener(e -> reset_3031());
96.     }
97.
98.     // tambah data
99.     private void tambahData_3031() {
100.         String nama = txtNama_3031.getText();
101.         String nim = txtNim_3031.getText();
102.         String prodi = txtProdi_3031.getText();
103.
104.         if (nama.equals("") || nim.equals("") ||
105.             prodi.equals("")) {
106.             JOptionPane.showMessageDialog(this, "Data belum
107.                 lengkap");
108.             return;
109.         }
110.
111.         Mahasiswa_2511533031 mhs_3031 =
112.             new Mahasiswa_2511533031(nama, nim, prodi);
113.
114.         listMahasiswa_3031.add(mhs_3031);
115.
116.         tampilData_3031();
117.
118.         txtNama_3031.setText("");
119.         txtNim_3031.setText("");
120.         txtProdi_3031.setText("");
121.     }
122.
123.     // tampil tabel
124.     private void tampilData_3031() {
125.         model_3031.setRowCount(0);
126.
127.         for (int i = 0; i < listMahasiswa_3031.size(); i++)
128.         {

```

```

128.
129.         model_3031.addRow(new Object[]{
130.             i + 1,
131.             listMahasiswa_3031.get(i).getNama_3031(),
132.             listMahasiswa_3031.get(i).getNim_3031(),
133.             listMahasiswa_3031.get(i).getProdi_3031()
134.         });
135.     }
136. }
137.
138. // hapus data
139. private void hapusData_3031() {
140.
141.     int baris = tabel_3031.getSelectedRow();
142.
143.     if (baris == -1) {
144.
145.         JOptionPane.showMessageDialog(this,
146.             "Pilih data dulu");
147.     }
148.     else {
149.
150.         listMahasiswa_3031.remove(baris);
151.
152.         tampilData_3031();
153.     }
154. }
155.
156. // mulai sorting
157. private void mulaiSorting_3031() {
158.
159.     areaLog_3031.setText("");
160.
161.     String pilihan =
162.         comboSort_3031.getSelectedItem().toString();
163.
164.     if (pilihan.equals("Insertion Sort")) {
165.
166.         insertionSort_3031();
167.     }
168.     else if (pilihan.equals("Selection Sort")) {
169.
170.         selectionSort_3031();
171.     }
172.     else {
173.
174.         bubbleSort_3031();
175.     }
176.
177.     tampilData_3031();

```

```

178.         JOptionPane.showMessageDialog(this,
179.             "Sorting selesai");
180.     }
181.
182.     // insertion sort
183.     private void insertionSort_3031() {
184.
185.         areaLog_3031.append("=== INSERTION SORT ===\n");
186.
187.         for (int i = 1; i < listMahasiswa_3031.size(); i++)
188.         {
189.             Mahasiswa_2511533031 key =
190.                 listMahasiswa_3031.get(i);
191.
192.             int j = i - 1;
193.
194.             while (j >= 0 &&
195.                 listMahasiswa_3031.get(j).getNama_3031()
196.                 .compareToIgnoreCase(key.getNama_3031()) > 0) {
197.
198.                 listMahasiswa_3031.set(j + 1,
199.                     listMahasiswa_3031.get(j));
200.
201.                 j--;
202.             }
203.
204.             listMahasiswa_3031.set(j + 1, key);
205.
206.             areaLog_3031.append(
207.                 "Langkah " + i + " : "
208.                 + tampilNama_3031() + "\n");
209.         }
210.     }
211.
212.     // selection sort
213.     private void selectionSort_3031() {
214.
215.         areaLog_3031.append("=== SELECTION SORT ===\n");
216.
217.         for (int i = 0; i < listMahasiswa_3031.size() - 1;
218.             i++) {
219.
220.             int min = i;
221.
222.             for (int j = i + 1; j <
223.                 listMahasiswa_3031.size(); j++) {
224.
225.                 if
226.                 (listMahasiswa_3031.get(j).getNama_3031()
227.                 .compareToIgnoreCase(

```

```

225.     listMahasiswa_3031.get(min).getNama_3031()) < 0) {
226.
227.         min = j;
228.     }
229. }
230.
231.     Mahasiswa_2511533031 temp =
232.         listMahasiswa_3031.get(i);
233.
234.     listMahasiswa_3031.set(i,
235.         listMahasiswa_3031.get(min));
236.
237.     listMahasiswa_3031.set(min, temp);
238.
239.     areaLog_3031.append(
240.         "Pass " + (i + 1) + " : "
241.         + tampilNama_3031() + "\n");
242.     }
243. }
244.
245. // bubble sort
246. private void bubbleSort_3031() {
247.
248.     areaLog_3031.append("=== BUBBLE SORT ===\n");
249.
250.     for (int i = 0; i < listMahasiswa_3031.size() - 1;
251. i++) {
252.         for (int j = 0; j < listMahasiswa_3031.size() -
253. i - 1; j++) {
254.             if
255. (listMahasiswa_3031.get(j).getNama_3031()
256.                 .compareToIgnoreCase(
257.                     listMahasiswa_3031.get(j +
258. 1).getNama_3031()) > 0) {
259.
260.                 Mahasiswa_2511533031 temp =
261.                     listMahasiswa_3031.get(j);
262.
263.                 listMahasiswa_3031.set(j,
264.                     listMahasiswa_3031.get(j + 1));
265.
266.                 listMahasiswa_3031.set(j + 1, temp);
267.             }
268.         }
269.
270.         areaLog_3031.append(
271.             "Pass " + (i + 1) + " : "
272.             + tampilNama_3031() + "\n");
273.     }
274. }

```



```

274.     // ubah list nama jadi string
275.     private String tampilNama_3031() {
276.
277.         String hasil = "[";
278.
279.         for (int i = 0; i < listMahasiswa_3031.size(); i++)
280.         {
281.             hasil +=
282.             listMahasiswa_3031.get(i).getNama_3031();
283.             if (i != listMahasiswa_3031.size() - 1) {
284.
285.                 hasil += ", ";
286.             }
287.         }
288.
289.         hasil += "]";
290.
291.         return hasil;
292.     }
293.
294.     // reset
295.     private void reset_3031() {
296.
297.         listMahasiswa_3031.clear();
298.
299.         tampilData_3031();
300.
301.         areaLog_3031.setText("");
302.
303.         txtNama_3031.setText("");
304.         txtNim_3031.setText("");
305.         txtProdi_3031.setText("");
306.     }
307.
308.     // main
309.     public static void main(String[] args) {
310.
311.         SwingUtilities.invokeLater(() -> {
312.
313.             new SortingGUI_2511533031().setVisible(true);
314.         });
315.     }
316. }

```

Program **SortingGUI_2511533031** digunakan untuk mengelola data mahasiswa sekaligus melakukan proses pengurutan data menggunakan algoritma Insertion Sort, Selection Sort, dan Bubble Sort berbasis GUI Java Swing. Program ini memiliki fitur menambah data mahasiswa, menghapus

data, melakukan sorting berdasarkan nama, serta menampilkan proses sorting pada area log. Data mahasiswa disimpan menggunakan ArrayList dengan objek dari kelas Mahasiswa_2511533031.

Pada bagian awal program terdapat deklarasi atribut seperti ArrayList<Mahasiswa_2511533031>, JTextField, JButton, JTable, DefaultTableModel, dan JTextArea yang digunakan untuk kebutuhan tampilan GUI dan penyimpanan data. Constructor SortingGUI_2511533031() berfungsi membangun tampilan program seperti form input, tabel data, tombol kontrol, combo box pilihan sorting, dan area log proses sorting.

Method tambahData_3031() digunakan untuk menambahkan data mahasiswa ke dalam ArrayList. Method ini mengambil input nama, NIM, dan prodi dari text field, kemudian membuat objek mahasiswa baru dan menambahkannya ke list. Setelah data masuk, method tampilData_3031() dipanggil untuk memperbarui isi tabel.

Method hapusData_3031() digunakan untuk menghapus data mahasiswa berdasarkan baris yang dipilih pada tabel. Jika belum ada data yang dipilih, program akan menampilkan pesan peringatan menggunakan JOptionPane.

Method mulaiSorting_3031() berfungsi menentukan algoritma sorting yang dipilih pengguna melalui ComboBox. Program akan menjalankan insertionSort_3031(), selectionSort_3031(), atau bubbleSort_3031() sesuai pilihan pengguna, lalu menampilkan hasil pengurutan pada tabel dan area log.

Method insertionSort_3031() melakukan pengurutan data mahasiswa menggunakan algoritma Insertion Sort dengan membandingkan nama mahasiswa menggunakan compareToIgnoreCase(). Setiap langkah sorting dicatat pada areaLog_3031.

Method selectionSort_3031() melakukan pengurutan menggunakan algoritma Selection Sort dengan mencari data terkecil

berdasarkan nama pada setiap iterasi, kemudian menukarnya dengan posisi saat ini.

Method bubbleSort_3031() melakukan pengurutan dengan membandingkan dua data bersebelahan dan menukarnya jika urutannya salah. Proses dilakukan berulang hingga seluruh data terurut.

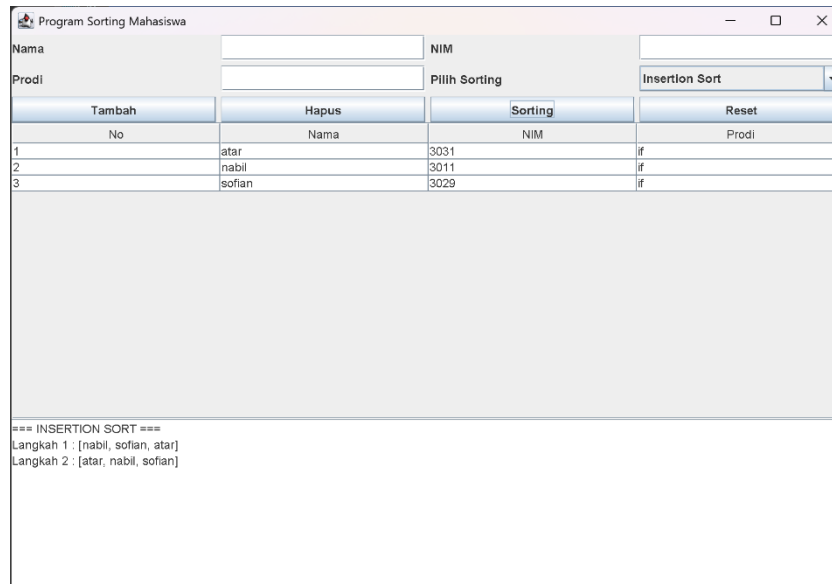
Method tampilNama_3031() digunakan untuk mengubah daftar nama mahasiswa menjadi bentuk string sehingga dapat ditampilkan pada area log langkah sorting.

Method reset_3031() berfungsi menghapus seluruh data mahasiswa, membersihkan tabel, area log, dan seluruh input text field sehingga program kembali ke kondisi awal.

Pada **method main()**, program dijalankan menggunakan `SwingUtilities.invokeLater()` agar tampilan GUI dapat berjalan dengan baik pada Java Swing.

B. Output Kode

1. Insertion Sort



No	Nama	NIM	Prodi
1	atar	3031	if
2	nabil	3011	if
3	sofian	3029	if

```

=== INSERTION SORT ===
Langkah 1 : [nabil, sofian, atar]
Langkah 2 : [atar, nabil, sofian]
  
```

2. Selection Sort

Program Sorting Mahasiswa

Nama NIM

Prodi Pilih Sorting Selection Sort

No	Nama	NIM	Prodi
1	atar	3031	if
2	nabil	3011	if
3	sofian	3029	if

=== SELECTION SORT ===
Pass 1 : [atar, nabil, sofian]
Pass 2 : [atar, nabil, sofian]

3. Bubble Sort

Program Sorting Mahasiswa

Nama NIM

Prodi Pilih Sorting Bubble Sort

No	Nama	NIM	Prodi
1	atar	3031	if
2	nabil	3011	if
3	sofian	3029	if

=== BUBBLE SORT ===
Pass 1 : [atar, nabil, sofian]
Pass 2 : [atar, nabil, sofian]